Samostatný text – Príklady algoritmov

Lekcia:	**Príklady algoritmov**
Modul/tematický celok:	**Základy programovania**
Téma/predmet:	**Počítačové systémy a siete**
Odbor:	**Elektrotechnik**

Riešenie každej úlohy sa skladá z postupnosti logických úsudkov. Počítačový program sa skladá z množiny príkazov, ktoré riešia zadanú úlohu. Pred písaním programu je potrebné nájsť algoritmus, postup riešenia úlohy, teda postupnosť krokov. Proces hľadanie algoritmu pre danú úlohu je algoritmizácia úlohy.

Výsledkom algoritmizácie úlohy je algoritmus. Algoritmus je presný systém pravidiel určujúci postup riešenia zadanej úlohy, pri ktorom získame z daných vstupov požadovaný výstup. Rozdiel medzi algoritmom a programom počítača je v tom, že program musí vyhovovať až do najmenších detailov pravidlám konkrétneho programovacieho jazyka. Aj najmenšia chyba môže viesť k nekontrolovateľnému správaniu počítača. Ten istý algoritmus môže byť realizovaný v rôznych programovacích jazykoch .

Algoritmizácia úlohy prebieha v troch etapách:

1. formulácia - slovné zadanie úlohy,
2. analýza - úloha sa zovšeobecňuje, určujú sa podmienky postupu,
3. zostavenie algoritmu - presné vyjadrenie logiky a postupu riešenia.

Algoritmus je konečná postupnosť dobre definovaných inštrukcií na splnenie určitej úlohy.
Algoritmy môžu byť zapísané (implementované) vo forme počítačových programov. Logická chyba v algoritme môže viesť k zlyhaniu výsledného programu.
Pojem algoritmu sa často ilustruje na príklade receptu, hoci algoritmy sú často oveľa zložitejšie. V algoritmoch sa často niekoľko krokov viacnásobne opakuje (iterácia), alebo ďalší postup závisí od aktuálneho stavu (vetvenie).
Na riešenie tej istej úlohy môže existovať niekoľko rôznych algoritmov s rôznymi postupnosťami inštrukcií. Rôzne algoritmy sa tiež môžu líšiť v množstve času a pamäte potrebných na splnenie úlohy.

Základné vlastnosti algoritmu

Rezultatívnosť (konečnosť)

Každý algoritmus musí skončiť po vykonaní konečného počtu krokov. Tento počet krokov môže byť ľubovoľne veľký podľa rozsahu a hodnôt vstupných údajov, ale pre každý jednotlivý vstup musí byť konečný. Postupy, ktoré túto podmienku nespĺňajú, sa môžu nazývať výpočtové metódy. Špeciálnym príkladom nekonečnej výpočtovej metódy je reaktívny proces, ktorý priebežne reaguje s okolitým prostredím.

Determinovanosť

Každý krok algoritmu musí byť jednoznačne a presne definovaný; v každej situácii musí byť úplne zrejmé, čo a ako sa má vykonať, ako má vykonávanie algoritmu pokračovať. Pretože bežný jazyk zvyčajne neposkytuje úplnú presnosť a jednoznačnosť vyjadrovania, boli pre zápis algoritmov navrhnuté programovacie jazyky, v ktorých má každý príkaz jasne definovaný význam. Vyjadrenie algoritmu v programovacom jazyku sa nazýva program.

Efektívnosť

Všeobecne požadujeme, aby algoritmus bol efektívny. Požadujeme, aby každá operácia vykonávaná algoritmom, bola dostatočne jednoduchá na to, aby mohla byť aspoň v princípe realizovaná v konečnom čase iba s použitím ceruzky a papiera.

Všeobecnosť

Algoritmus nerieši jeden konkrétny problém, napríklad ako vypočítať 3×7, ale rieši všeobecnú triedu obdobných problémov, teda ako vypočítať súčin dvoch celých čísel.

Algoritmické metódy

Algoritmické postupy sa dajú zobraziť mnohými spôsobmi. Dôležité je, aby sme si po čase vedeli spomenúť, čo sme zaznamenali, a aby nášmu záznamu rozumeli aj ostatní. Pre zobrazenie algoritmu používame rôzne metódy. Patrí k nim:

1. textom, ako formálne popísaný priebeh pracovného postupu vo forme textu členeného na paragrafy,
2. dátovo orientované diagramy HIPO spojujúce v grafickom vyjadrení funkčné členenie problému, štruktúry dát a pracovných postupov pri rôznom stupni podrobnosti,
3. štruktúrované diagramy zamerané predovšetkým na zobrazenie postupností operácií v úlohe, s využitím štandardných algoritmických štruktúr,

4. štruktúrované diagramy vo forme stromového diagramu, v ktorom sa procesy postupne členia až na najnižšiu úroveň,
5. vývojové diagramy toku dát a vývojové diagramy programu zobrazujúce predovšetkým postupnosť operácií v systéme a v jednotlivých úlohách,
6. rozhodovacie tabuľky využívané zvlášť pri zložitých rozhodovacích procesoch v algoritmoch programu,
7. samostatným programovacím jazykom,
8. slovne.

Grafické zobrazenie algoritmu úlohy voláme vývojové diagramy.

Vývojové diagramy popisujú algoritmus programu grafickými značkami.

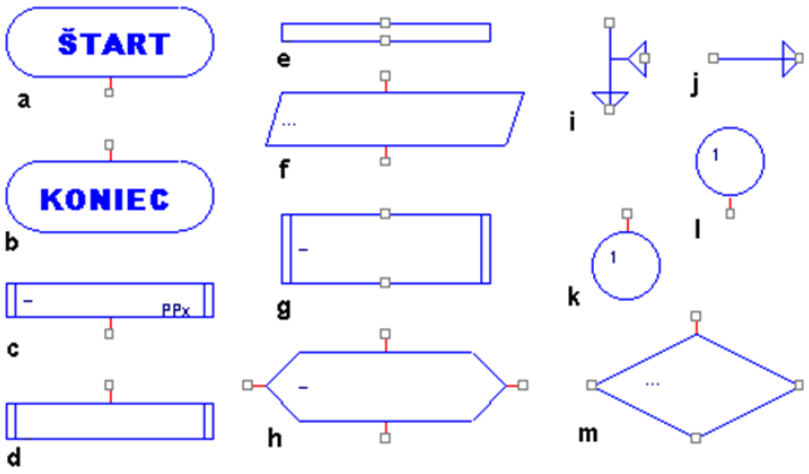
Zápis algoritmov

a) prirodzeným jazykom, ale s kľúčovými slovami kvôli lepšej zrozumiteľnosti
   - výhody – ľahko čitateľný, každý mu rozumie,
   - nevýhody – nejednoznačnosť, nízka prehľadnosť.

b) Vývojové diagramy
   - výhody – jasný tok riadenia, viaceré úrovne podrobností, dvoj rozmernosť,
   - nevýhody – zápis, nedá sa spracovať priamo v počítači.

Grafické značky vývojových diagramov

Grafické značky vývojových diagramov sú zobrazené na obrázku č.01



Obrázok č. 01 <Grafické značky vývojových diagramov >

Význam jednotlivých značiek:

- **a -** začiatok algoritmu
- **b -** koniec algoritmu
- **c -** začiatok podprogramu
- **d -** koniec podprogramu
- **e -** jednoduchý príkaz (viac za sebou vytvárajú blok príkazov)
- **f -** čítaj resp. vypíš (aj vytlač)
- **g -** volanie podprogramu
- **h -** riadený cyklus s konkrétnym počtom opakovaní
- **i, j -** spojenie dvoch ciest resp. smerovanie programu
- **k -** odskok na návestie
- **l -** návestie na ktoré smeruje skok
- **m -** rozhodovací blok s jedným vstupom a viacerými výstupmi.

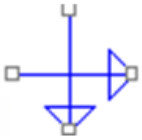
Jednotlivé značky sú spojené spojnicami

- vodorovne,
- zvisle.

Môžu, ale nemusia mať šípku, ktorá určuje smer vykonávania jednotlivých činností

Spojnice sa vo vývojovom diagrame môžu:

1. križovať, viď obrázok č.02,
2. spájať, viď obrázok č.03.



Obrázok č. 02 Križovanie spojníc

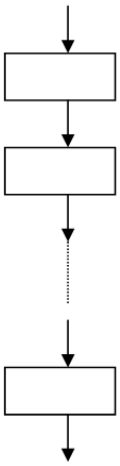


Obrázok č. 03 Spájanie spojníc

Ľubovoľný výpočtový proces môžeme opísať pomocou týchto riadiacich štruktúr :

1. sekvenčnosť,
2. binárne vetvenie,
3. cyklus.

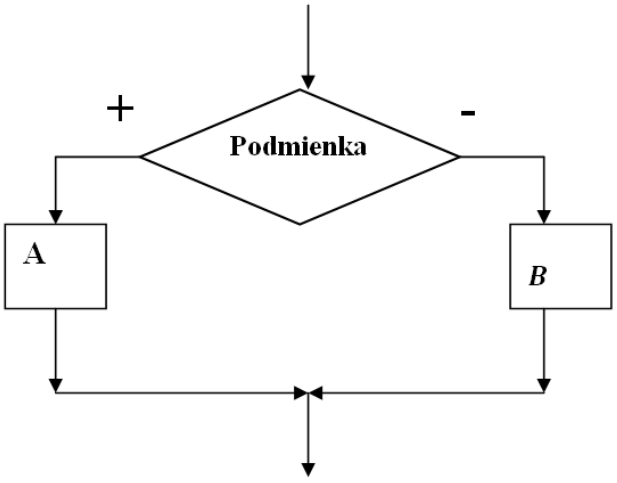
Sekvenčnosť – akcie sú vykonávané za sebou v takom poradí, ako sú napísané, ak nie je explicitne dané iné poradie. Viď obrázok č. 04



Obrázok č. 04 Sekvenčnosť vývojového diagramu

Binárne vetvenie – výber jednej z dvoch akcií podľa toho, či je nejaká podmienka splnená alebo nie. Je prípustné, aby jedna z dvoch uvažovaných akcií bola prázdna, čiže akcia bez operačného účinku.

Viď obrázok č. 05



Obrázok č. 05 Binárne vetvenie vývojového diagramu

Ak je podmienka splnená, vykoná sa akcia A, ak nie, vykoná sa akcia B

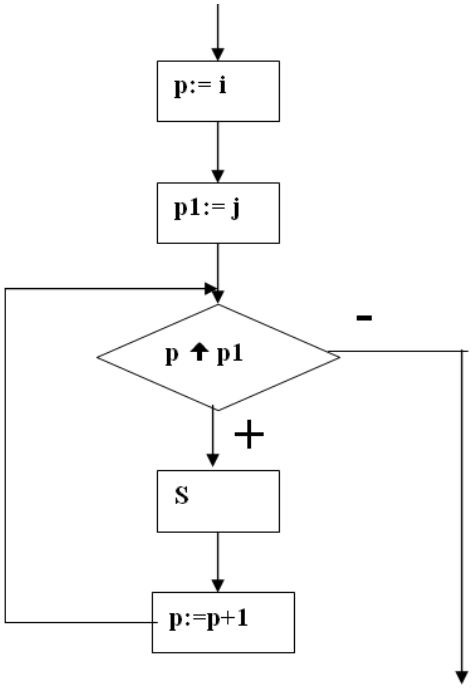
Cyklus – opakované vykonávanie príkazu alebo sekvencie príkazov s explicitne alebo implicitne udaným počtom opakovaní.

Typy cyklov

Poznáme tri základné typy cyklov:

1. cyklus s pevným počtom opakovaní,
2. cyklus s podmienkou na začiatku,
3. cyklus s podmienkou na konci.

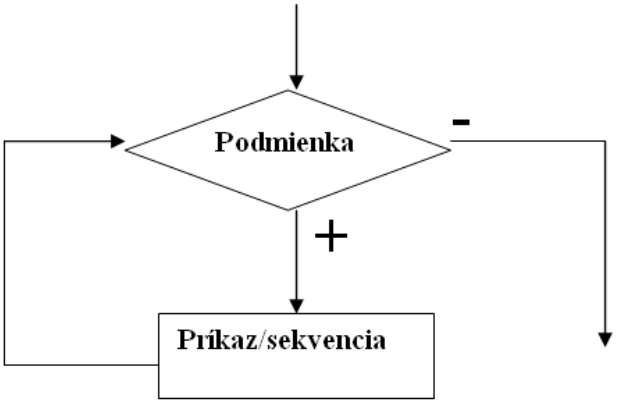
Cyklus s pevným počtom opakovaní – počet opakovaní príkazu, príkazov je daný explicitne (pevne), ako je znázornené na obrázku č. 06



Obrázok č. 06 Cyklus s pevným počtom opakovaní

Pre p od i až po j opakuj S

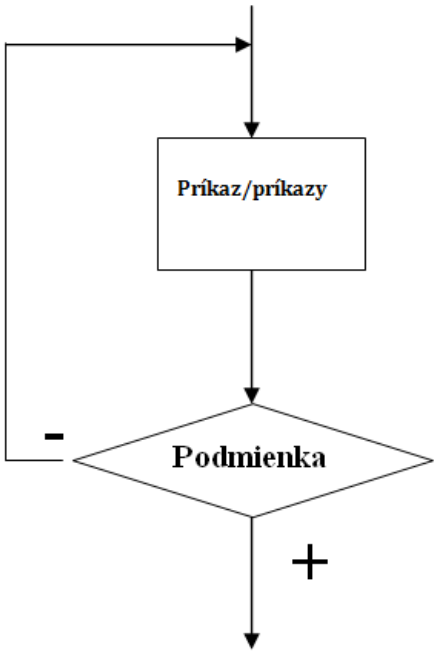
Cyklus s podmienkou na začiatku – počet opakovaní príkazu, príkazov je explicitne daný s testom podmienky na začiatku pred časťou, ktorá sa má vykonávať, ako je znázornené na obrázku č. 07.



Obrázok č. 07 Cyklus s podmienkou na začiatku

Pokiaľ je splnená podmienka, opakuj príkaz, sekvenciu. Cyklus nemusí zbehnúť ani raz.

Cyklus s podmienkou na konci – počet opakovaní príkazu, príkazov je daný implicitne s testom po časti, ktorá sa má opakovať, ako je znázornené na obrázku č. 08



Obrázok č. 08 Cyklus s podmienkou na konci

Opakuj A, až pokiaľ sa B nesplní, alebo B neplatí. Cyklus je vždy vykonaný aspoň raz.